

Finding Missed Code-Size Optimizations in Compilers Using LLMs

Authors: Davide Italiano, Chris Cummins

Venue: Meta AI (FAIR) — January 2025



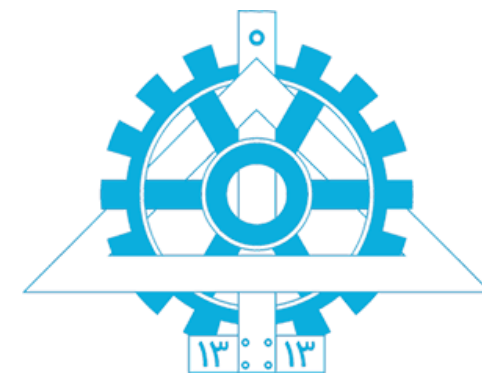
Presenter: Mohammad Amanlou

Email: MohammadAmanlou2@gmail.com

Date: 10rd June

Course: Programming Languages and Compilers

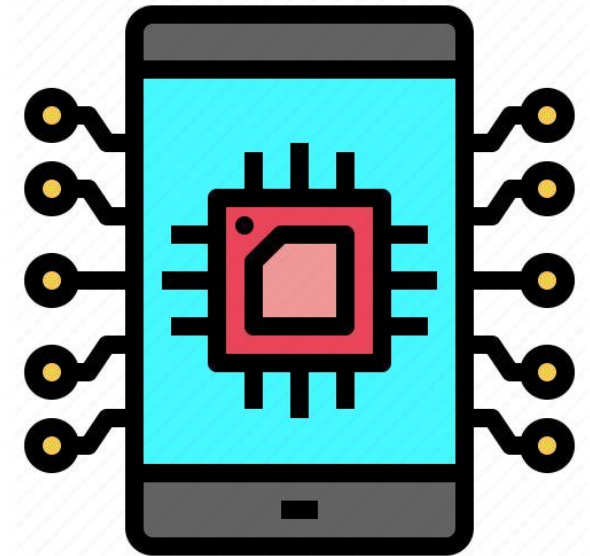
Instructor: Dr. Tavassoli



Motivation: Why Code Size Matters?

- **Mobile & Embedded Constraints**

- Flash / cache budgets are tight: 10 KB extra can tip an MCU over its limit.
- Larger binaries → longer cold-start, higher energy drain, slower OTA updates.
- App-store rules increasingly penalise bloated packages (e.g. Google Play's 100 MB cap).



Motivation: Why Code Size Matters?

- **Hidden Bloat Accumulates**

- Modern builds pull in hundreds of optimisation passes and libraries.
- One missed pass can replicate across thousands of functions.
- Small per-function overhead $\times$ large codebase $\approx$ megabytes of waste.



Motivation: Why Code Size Matters?

- **The Testing Gap**

- 20 + years of compiler fuzzing $\approx$ functional-correctness only.
- Size-centric testing tools are rare, language-specific, and hand-crafted.
- Result: many “correct-but-bloated” binaries ship unnoticed.



Motivation: Why Code Size Matters?

- **Real-World Impact**

- Internal Meta study: fixing a single size bug cut Android launch time by **8 %**.
- Embedded firmware vendors pay \$\$ per kilobyte in flash cost.
- Smaller binaries mean shorter test cycles and less CI storage.



Motivation: Why Code Size Matters?

- **The LLM Opportunity**

- Off-the-shelf models can now emit syntactically valid C/Rust/Swift on demand.
- 150-line Python harness + LLM = fully automatic size fuzzing.
- Portable: swap prompt & compiler command → new lang



LLMS



EDITABLE STROKE



Related Works

Grammar-based random generators

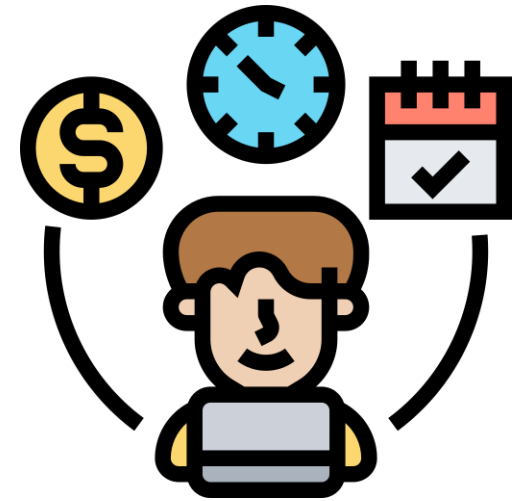
- *CSmith* (2011) → > 40 K lines of handwritten C++
- Guarantees syntactic + semantic correctness
- Costly to extend when the language evolves

Mutation testing before LLMs

- *EMI / Equivalence-Modulo-Inputs* (Le 2014)
- Requires seed libraries + static analyses
- Hand-crafted mutation logic, language-specific

Blind spot

Tools excel at catching functional mis-compilations, but say little about code-size bloat.



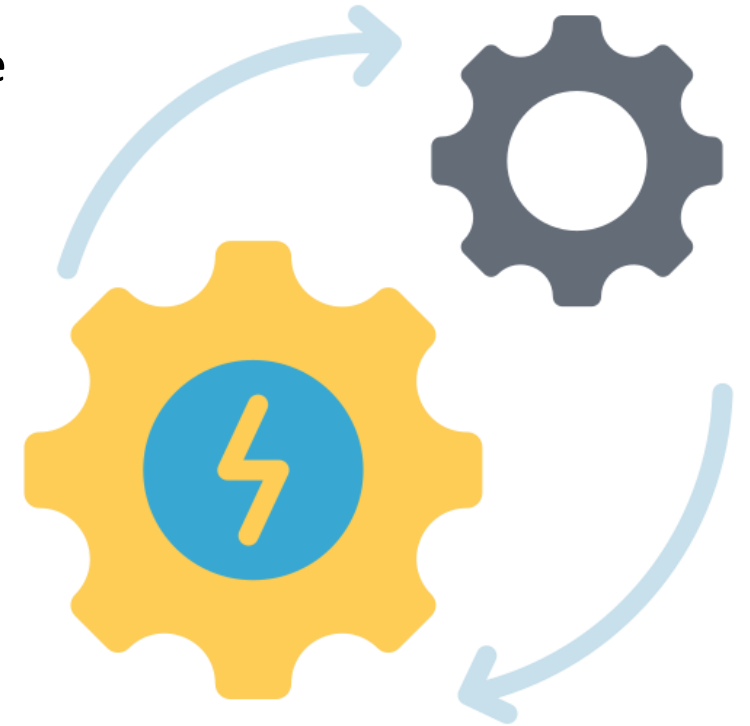
Related Works

Deep RNN / LSTM approaches

- Cummins 2018, Liu 2019 → needed ~20 tries for one compilable snippet;
- weak grasp of syntax/semantics.

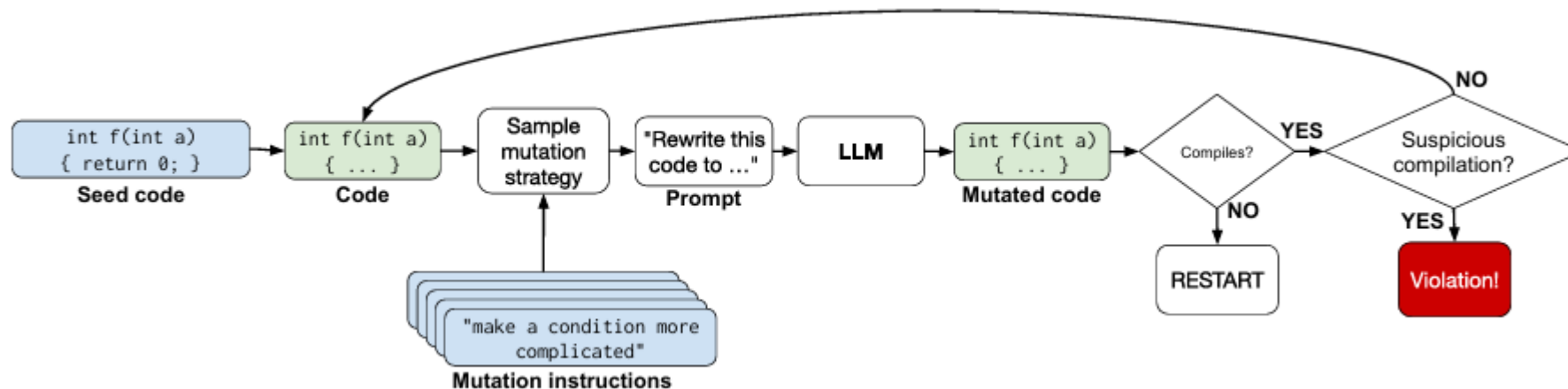
LLM-based *Fuzz4All* (Xia 2024)

- Ingests docs + examples to create tests for functional correctness
- Still depends on external documentation
- No focus on missed size optimisations
- **Key gap:** Despite ML progress, “**correct-but-bloated**” binaries remain largely untested.



Pipeline at a Glance

- **Seed program**
- **Randomly pick a mutation instruction**
 - (e.g. “add nested loop”).
- **LLM rewrites code**
 - via prompt → *mutated snippet*.
- **Compile & run *one* differential-testing strategy.**
 - ✓ No issue → next mutation
 - ⚠ Size anomaly → **Violation** logged.
 - Up to 10 mutations per episode, then restart.



START

```
int f(int a) {
    return 0;
}
```

Seed program.

STEP 1 "add pointers code"

```
- int f(int a) {
+ int f(int* a) {
- return 0;
+ return *a;
}
```

✔ Nothing of
interest, proceed.

STEP 2 "add a nested loop with a complex condition and a statement inside"

```
int f(int* a) {
- return *a;
+ int sum = *a;
+ for (int i = 0; i < 10; i++) {
+     if (i % 2 == 0) {
+         for (int j = 0; j < 5; j++) {
+             if (j > 2 && sum > 0) {
+                 sum += i + j;
+             }
+         }
+     }
+ }
+ return sum;
}
```

✔ Nothing of
interest, proceed.

STEP 3 "add a nested loop with a complex condition and a statement inside"

```
int f(int* a) {
    int sum = *a;
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) {
            for (int j = 0; j < 5; j++) {
                if (j > 2 && sum > 0) {
                    sum += i + j;
                }
                for (int k = 0; k < 3; k++) {
                    if ((k * j) % 2 != 0 && i > 5) {
                        sum -= k;
                    }
                }
            }
        }
    }
    return sum;
}
```

✔ Nothing of
interest, proceed.

STEP 4 "make a condition more complicated"

```
int f(int* a) {
    int sum = *a;
    for (int i = 0; i < 10; i++) {
-     if (i % 2 == 0) {
+     if (i % 2 == 0 && (i > 3 || *a < 5)) {
        for (int j = 0; j < 5; j++) {
            if (j > 2 && sum > 0) {
                sum += i + j;
            }
            for (int k = 0; k < 3; k++) {
                if ((k * j) % 2 != 0 && i > 5) {
                    sum -= k;
                }
            }
        }
    }
    return sum;
}
```

✔ Nothing of
interest, proceed.

STEP 5 "add a nested conditional statement with a non trivial condition and a statement inside"

```
int f(int* a) {
    int sum = *a;
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0 && (i > 3 || *a < 5)) {
            for (int j = 0; j < 5; j++) {
                if (j > 2 && sum > 0) {
                    sum += i + j;
                }
                for (int k = 0; k < 3; k++) {
                    if ((k * j) % 2 != 0 && i > 5) {
                        sum -= k;
                    }
                    if (k > 1 && j < 4 && (sum % (i + 1)) == 0) {
                        sum += k * j;
                    }
                }
            }
        }
    }
    return sum;
}
```



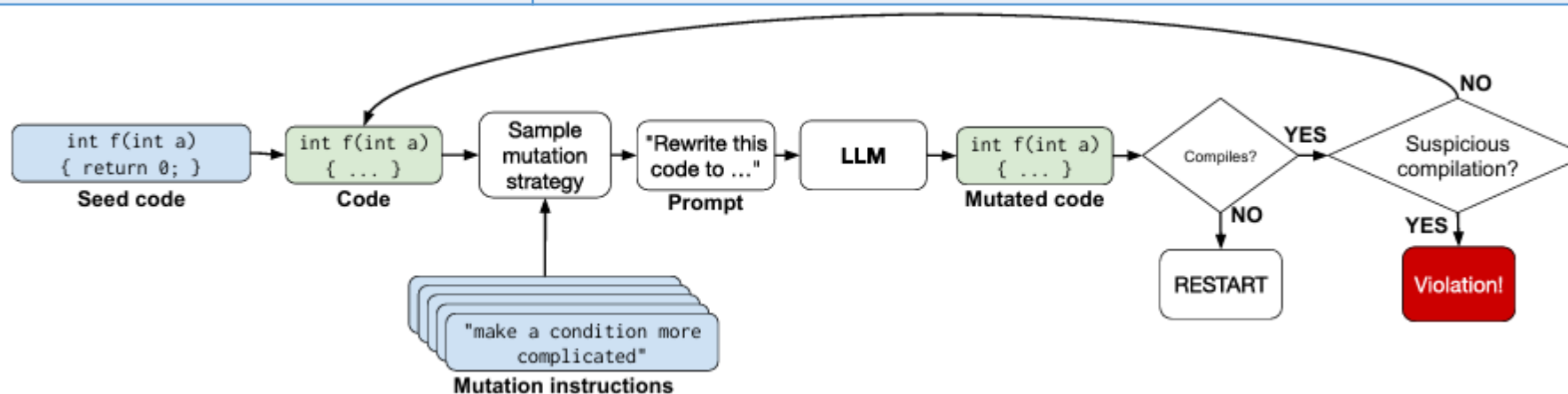
Potential regression found!
This code leads to a 36% larger binary when
compiled using GCC 15 vs GCC 14.

GCC Bug #117128

Bisected to revision r14-2161-g237e83e2158a3d.
Identified as a problem with shrink-wrapping.

More Details

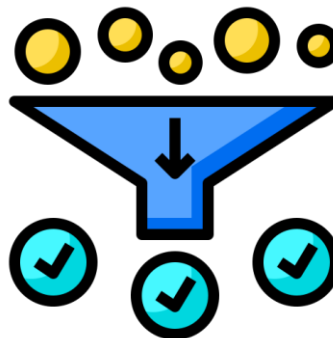
Step	What Happens
Prompt Template	"Given the following $\{language\}$ program, please $\{instruction\}$: ..."
Mutation Instructions	Control-flow inserts · pointer / aggregate code · conditional rewrites · dead-code variants
LLM Model	Llama 3.1 — 70 B or 405 B parameters (off-the-shelf)
Compile Check	If the output fails to compile → restart the episode
Dynamic Guardrails	- Coverage counters verify that "dead" code is never executed - Time-out kills infinite loops



Detecting False Positives

Why We Need False-Positive Filtering?

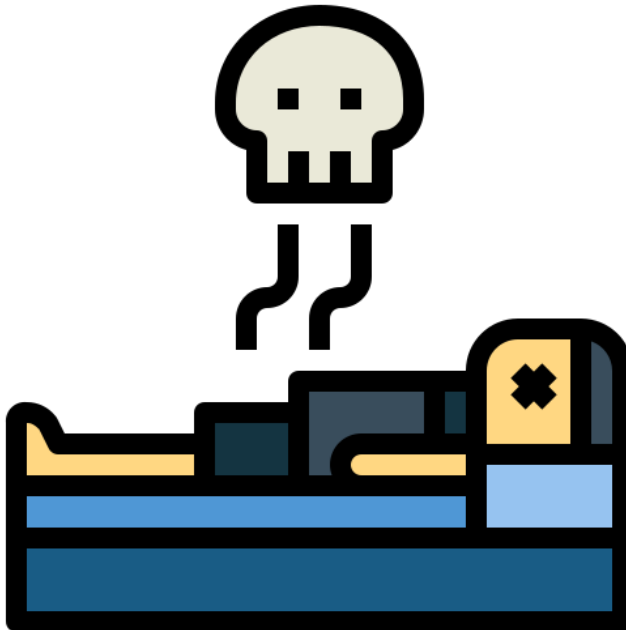
- LLM $\neq$ perfect control
 - Model may output illegal or semantically wrong code.
- Goal: report *real* missed-optimization bugs, **not** self-inflicted errors.
- **Solution:** three complementary filters that catch bad samples *before* they reach the bug tracker.



Detecting False Positives

Filter #1: Dead-Code Verification

1. Auto-wrap mutated function in a tiny
2. Run with coverage counters.
3. Confirm counters for injected lines stay zero.
4. Signature corrupted? → compile fails → restart.



Detecting False Positives

Filter #2: Sanitizers

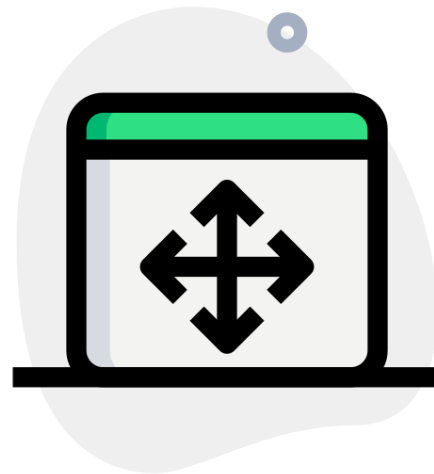
- **Tools used**
 - **ASan + UBSan** (LLVM) → memory & UB errors.
 - **CompCert interpreter** (when available) → extra UB sweep.
- **Impact:** < 10 % of candidates rejected, huge drop in false positives.



Detecting False Positives

Filter #3: Monotonic-Size Heuristic

- Undefined-behavior often lets the optimiser *delete* code. Our mutations *add* complexity → size should grow or stay equal.
Shrink ⇒ likely UB ⇒ discard sample.
- **Fast & cheap**; first gate in the pipeline—higher false-negative rate, but near-zero cost.



Dead-Code Differential Testing

Key idea

Add *only* truly dead code to a constant-return seed; program semantics stay identical.

Procedure

Disable all mutations except “insert dead code,” rebuild twice with the *same* compiler, compare.

Trigger rule

Any size difference $\Rightarrow$ compiler failed to delete dead code $\Rightarrow$ missed optimisation.

Perks

No second compiler or extra flags needed; lightning-fast to run.

Typical win

Catches regressions in passes like *DCE*, *Value-Range Analysis*, *Constraint Elimination*.



Optimisation-Pipeline Differential Testing

Key idea

Pipelines such as -Oz / opt-level=z promise *smallest possible* code.

Procedure

Compile once with size-centric pipeline (-Oz) and again with a performance-centric one (-O3/-Os).

Trigger rule

If non-Oz binary $\geq 5\%$ *smaller* than Oz $\Rightarrow$ violation.

Perks

Single compiler, no historical versions; exposes internal pipeline gaps (e.g., loop unroll only in -O3).

Real bug

LLVM 111571: -O3 shrank a loop to const, -Oz left it intact.



Single-Compiler Differential Testing

Key idea

Detect *regressions* (or accidental gains) over time.

Procedure

Compare nightly/trunk build with last stable release(s).

Trigger rule

Size grows in new version $\Rightarrow$ potential regression logged.

Perks

Great for CI; pinpoints when an optimisation pass back-slides.

Workflow boost

Automated Git bisection can name the first bad commit.



Multi-Compiler Differential Testing



Key idea

Pit different compilers (e.g., Clang vs GCC) against the same source.

Procedure

Build with equivalent flags; measure output sizes.

Trigger rule

If one binary $\geq 10\%$ larger, flag that compiler as missing an optimisation present in the other.

Perks

Language-agnostic, discovers “feature gaps” across back-ends; richest source of violations in experiments ($\approx 30\%$).

Example

GCC 116868: Clang folded a std-vector to a constant, GCC did not—now fixed upstream.

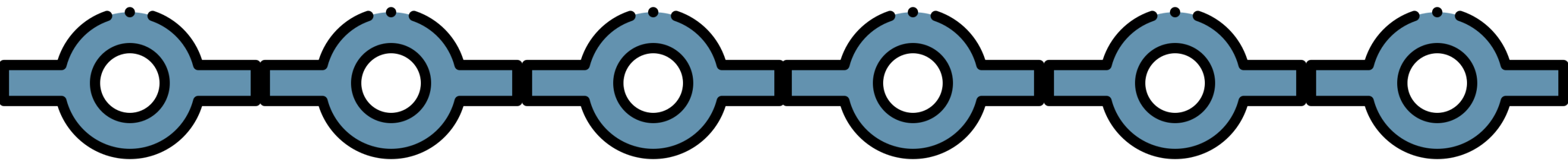
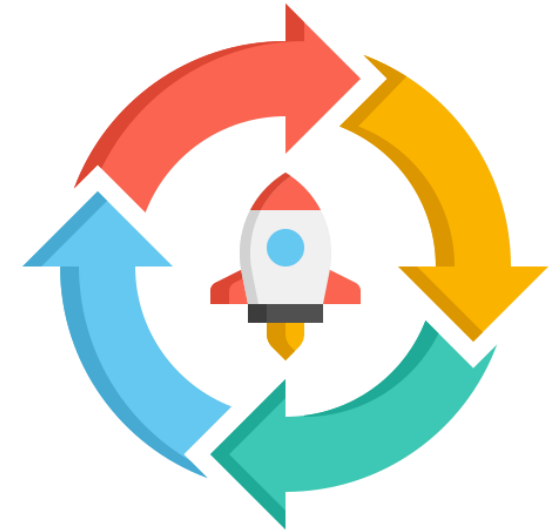


Release Screening

Test the failing program on several public releases of the same compiler (e.g., GCC 12 → 13 → 14)

Interpretation: If *exactly* the same versions fail, you probably have one bug;
partial overlap → likely two distinct bugs.

Benefit: Cheap first filter that groups many reports automatically.



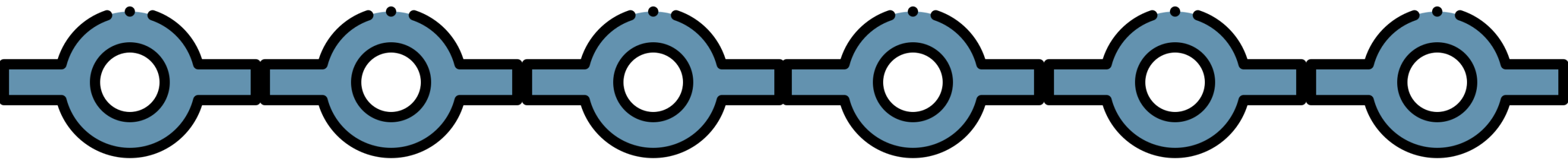
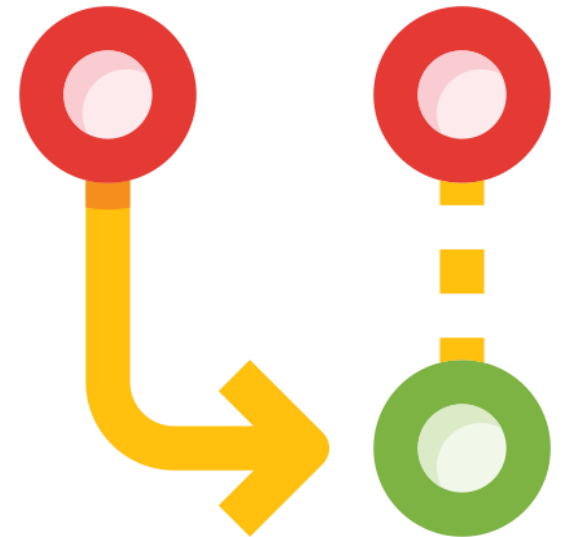
Commit Screening / Bisection

Setup: For a regression, we know a “last-good” revision.

Bisect: Binary-search Git history to find the first bad commit.

Duplicate rule: Two programs → two *different* culprit commits ⇒ two bugs;
same commit ⇒ collapse into one report.

Outcome: Sharper bug tracker, faster developer turnaround.



Experimental Setup

The experiment was tailored for the C language, and the results obtained with two different LLMs are as follows:

Metric	70 B (Total)	405 B (Total)
Programs generated	28 k +	8.4 k
Compilable %	96.4 %	94.7 %
Violations found	3 651	1 258
Top strategy	Multi-compiler: 30.8 % violations	Multi-compiler: 35.8 % violations
Mean Time	4.18 s	12.23 s



Thank You!

